

Bibex · the page's own bib, worked through a citation workbench

1 One file is yours, the rest is rebuilt

A MIXED plugin, like `display/`: one file a person rules, and files a rebuild may replace.

```
<page>/bibex/
  <stem>.bib          PRIMARY . the page's citation store . committed
                        header says: edits welcome, refresh only APPENDS
  <stem>-bib.html     DERIVED . the workbench view, regenerated freely
  .board-refs.bbl     DERIVED . the formatted references, for the word export
flat page fallback: <board>/bibex/
```

The page's `.bib` is yours to write and is never overwritten, while the card view and the formatted references are rebuilt for you at any time. The `.board-refs.bbl` comes from no door at all: the word export runs `cli/refs.py` over this bib whenever the bib is newer, so the `.docx` prints the in-text labels and the References list. Filling looks a cited key up in the paper's `0-*.bib`, searching upward, and copies the whole entry into the page bib. The paper file is read and never written. A refresh scans the page's keys again, brings in what can now be found, and draws the view again. An entry already in the page bib is never overwritten and never deleted, because a person may have edited it. An entry no longer cited on the page stays, marked **not cited on this page**, so cutting a sentence never quietly drops its source. A cite inside a code fence or backticks is stripped before the scan, because a figure showing the syntax is a picture, not a citation.

2 What a person can do to an entry, and what the machine may not

What a person does on a card: check, mark, fix, add.

```
[x] checked . JL 260815  [undo]    the tick writes verified = {JL 260815}
[ ] unchecked [ I checked this] INTO the entry . travels with it,
                                no sidecar to drift

Scholar . DOI . URL          the links the checking happens through
raw bibtex . edit [save]     fix a field, replace in place
missing key  Scholar        find it, paste its bibtex, it lands
  [ paste]  done             verbatim in the page bib
add a citation               same pen, for uncited sources
```

A person ticks, fixes, and pastes, and the machine only copies text whole and never writes a reference itself. The `answers` one question: did I check this link myself? Nobody but a person clicks it, the field records who and when, and `undo` takes it away. The `edit` buttons check the shape only, one balanced entry with a key. They refuse a repeated key unless the `save` says outright that it replaces the old one. They never write, finish, or fix the text itself, and that is the line between a pen and an author. The `box` also takes a `LINK`: a DOI, an arXiv link, Scholar's Cite to BibTeX link, or a paper URL. The bibtex is then fetched `WHOLE` from that source into the box. Fetching is copying, so the fetched text sits in the box for review, and landing it stays the person's second

click. A fetched key you cannot use, such as doi.org’s URL-as-key, is renamed to surname plus year by rule, because a key is a local handle and not information about the paper. The whole raw `.bib` opens right below the cards, with its path printed. The file is the person’s, and editing it by hand is allowed. This page’s own file holds one entry, JL’s KDD paper [Luo et al., 2026]. It is cited here so the page draws the inline link and the References block that the binding makes.

3 How the tab gets on screen

The tab: framed the way the slide tab is, from a registry entry, a HEAD probe, and an iframe, with the view carrying its own buttons.

```
82-plugin-exports.js --POST--> /_board/bibex --> refresh + view
tab.url()  names bibex/<stem>-bib.html . HEAD hit > frame it
the view's own buttons POST bibex-verify / bibex-entry, then reload
clicking the  tab while it is open > the refresh door again
```

The tab frames a page the server wrote, and the buttons inside that page talk to the server on their own. Clicking a tab that is already open rebuilds what that tab shows, so the second click on is the refresh: it draws the view again and appends any import the paper bib can now resolve, and it touches no entry already there.

References

Junjie Luo, Xuzhe Zhi, Rui Han, Abhimanyu Kumbara, Mansur E Shomali, Anand K Iyer, Ritu Agarwal, and Guodong Gordon Gao. Eventglucose: A benchmark for continuous glucose monitoring forecasting with behavioral event context. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 9510–9521, 2026.